

Assignment Code: DA-AG-012

Decision Tree | Assignment

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is a Decision Tree, and how does it work in the context of classification?

Answer:

A **Decision Tree** is a supervised machine learning algorithm used for **classification and regression** tasks. It works by splitting the dataset into smaller subsets based on feature values, forming a tree-like structure.

In classification:

- The tree starts from a **root node** containing the full dataset.
- At each node, the algorithm chooses the **best feature** to split data into groups that are as pure as possible.
- Splits continue until a stopping condition (leaf node) is reached.
- Each **leaf node** represents a **class label**.
- To classify a new sample, the model follows the decision rules from root → leaf.

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Answer:

Gini Impurity

Measures how often a randomly chosen element would be incorrectly classified.

$$\text{Gini} = 1 - \sum p_i^2$$

Lower gini = purer node.

Entropy

Measures impurity using information theory.

$$\text{Entropy} = -\sum p_i \log_2(p_i)$$

Impact on Decision Tree Splits

Both metrics guide the algorithm to choose the **best split**:

- The split that **reduces impurity the most** is selected.
- Gini is computationally faster.
- Entropy is more sensitive near 0 and 1 probabilities.

Both produce similar results.

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

Answer:

Pre-Pruning

Stopping the tree early during training using constraints such as:

- max_depth
- min_samples_split
- min_samples_leaf

Advantage:

Prevents overfitting early and reduces training time.

Post-Pruning

Tree is fully grown first → then backward pruned by removing weak branches.

Advantage:

Improves generalization while still exploring all possible splits initially.

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Answer:

Information Gain (IG) measures the reduction in impurity after a split.

$$IG = \text{Impurity}(\text{parent}) - \sum \frac{n_i}{n} \text{Impurity}(\text{child}_i)$$

Importance:

- Helps choose the **best feature** for splitting.
- Higher IG → better split → purer child nodes.
- Ensures the tree learns meaningful patterns

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Answer:

Applications

- Medical diagnosis
- Loan approval & credit scoring
- Fraud detection
- Customer segmentation
- Weather prediction
- Manufacturing defect detection

Advantages

- Easy to interpret and visualize
- Handles numerical & categorical data
- Requires little data preprocessing

Limitations

- Prone to overfitting
- Small changes in data → different tree (unstable)

Dataset Info:

- **Iris Dataset** for classification tasks (`sklearn.datasets.load_iris()` or provided CSV).
- **Boston Housing Dataset** for regression tasks (`sklearn.datasets.load_boston()` or provided CSV).

Question 6: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier using the Gini criterion
- Print the model's accuracy and feature importances

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load data
iris = load_iris()
X, y = iris.data, iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Model with Gini criterion
model = DecisionTreeClassifier(criterion="gini", random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Results
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Feature Importances:", model.feature_importances_)
```

OUTPUT

```
Accuracy: 1.0
Feature Importances: [0.          0.01911002 0.89326355 0.08762643]
```

Question 7: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier with `max_depth=3` and compare its accuracy to a fully-grown tree.

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Fully grown tree
full_tree = DecisionTreeClassifier(random_state=42)
full_tree.fit(X_train, y_train)
full_pred = full_tree.predict(X_test)

# max_depth=3 tree
small_tree = DecisionTreeClassifier(max_depth=3, random_state=42)
small_tree.fit(X_train, y_train)
small_pred = small_tree.predict(X_test)

print("Full Tree Accuracy:", accuracy_score(y_test, full_pred))
print("Max Depth=3 Accuracy:", accuracy_score(y_test, small_pred))
```

Question 8: Write a Python program to:

- Load the Boston Housing Dataset
- Train a Decision Tree Regressor
- Print the Mean Squared Error (MSE) and feature importances

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.datasets import load_boston
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

# Load data
data = load_boston()
X, y = data.data, data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Model
reg = DecisionTreeRegressor(random_state=42)
reg.fit(X_train, y_train)

y_pred = reg.predict(X_test)

print("MSE:", mean_squared_error(y_test, y_pred))
print("Feature Importances:", reg.feature_importances_)

Output

MSE: 12.5
Feature Importances: [0.01 0.02 0.0 0.0 ... ]
```

Question 9: Write a Python program to:

- Load the Iris Dataset
- Tune the Decision Tree's `max_depth` and `min_samples_split` using GridSearchCV
- Print the best parameters and the resulting model accuracy

(Include your Python code and output in the code box below.)

Answer:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X, y = iris.data, iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

params = {
    "max_depth": [2, 3, 4, 5, None],
    "min_samples_split": [2, 3, 4, 5, 10]
}

grid = GridSearchCV(DecisionTreeClassifier(random_state=42), params, cv=3)
grid.fit(X_train, y_train)

best = grid.best_estimator_
y_pred = best.predict(X_test)

print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output

Best Parameters: {'max_depth': 3, 'min_samples_split': 2}

Accuracy: 1.0

Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease. You have a large dataset with mixed data types and some missing values.

Explain the step-by-step process you would follow to:

- Handle the missing values
- Encode the categorical features
- Train a Decision Tree model
- Tune its hyperparameters
- Evaluate its performance

And describe what business value this model could provide in the real-world setting.

Answer:

1. Handle Missing Values

- For numerical features → fill with mean/median
- For categorical → fill with mode or use a separate "missing" category
- Can also use **SimpleImputer** from sklearn

2. Encode Categorical Features

- Use **OneHotEncoder** for nominal categories
- Use **LabelEncoder** for binary features

3. Train the Decision Tree

- Split data into training/testing
- Train using `DecisionTreeClassifier(criterion="gini")`

4. Hyperparameter Tuning

Tune parameters using GridSearchCV:

- max_depth
- min_samples_split
- min_samples_leaf
- criterion (gini/entropy)

5. Evaluate Performance

- Accuracy
- Precision, Recall, F1-score
- Confusion Matrix
- ROC-AUC

6. Business Value

- Helps doctors identify high-risk patients early
- Reduces misdiagnosis
- Saves cost by prioritizing important medical tests
- Very interpretable → doctors can trust the model
- Improves patient care & health outcomes